

The Full-Stack Roadmap

From “comfortable with the basics” to job-ready — a 12-week intensive plan

Built for Shantanu — using a real creative-studio job description (DixonBaxi) as one worked example of what “full-stack” means in practice, generalised into a stack-agnostic roadmap for product/SaaS-style full-stack roles.

How to Use This Document

This isn't a syllabus to read — it's a set of checkpoints to pass. Each stage below gives you:

- **Skill targets**, organised by layer (frontend / backend / data / practices), scoped to what a hiring manager actually checks for at that level.
- **Principles**, the reasoning behind the skill — the thing that separates someone who copied a tutorial from someone who understands why the code is shaped that way.
- **A checkpoint**, a concrete, buildable increment on one running project (your “spine project” — see below). If you can't do the checkpoint, you're not ready to move on, regardless of how the calendar looks.
- **Self-check questions**, the kind a technical interviewer or senior colleague would actually ask. Answer them out loud, unaided, before you consider a stage done. If you have to look it up, that's your revision list.

A note on pace: 12 weeks intensive is realistic for you specifically because you're not starting from zero — you're coming in with professional C#/.NET experience, Azure DevOps CI/CD exposure, a React/TypeScript dashboard you built for a live trading system, and Python data work. That's genuine engineering context most “junior” candidates don't have. This plan assumes roughly 15–20+ focused hours a week. If a week slips, don't compress the later stages to compensate — extend the timeline instead. A slower roadmap you actually complete beats a fast one you abandon at week 7.

A note on AI tools: use them to scaffold boilerplate and explore unfamiliar APIs, but in Stages 1–2 write the core logic (the auth flow, the schema, the state management) yourself first, even slowly, even badly — then compare against what an AI suggests. The muscle you're building is judgement about generated code, not just output speed. By Stage 3–4 you should be leaning on AI more heavily and reviewing it as critically as you'd review a colleague's PR.

Where You Are Now

You self-assessed as “comfortable with the basics, a few personal projects shipped.” Cross-referenced against what you've actually built, that's a fair but slightly conservative read:

- **Genuine strengths**: professional experience shipping and maintaining code (C#/.NET at Cognizant, with real CI/CD pipeline ownership in Azure DevOps), a React + TypeScript control dashboard built for a live trading system, and applied Python/ML work (LSTM models, backtesting, MLflow). That's real evidence of state management, working with live/streaming-ish data, and end-to-end delivery.
- **Likely gaps (typical for this profile)**: you've leaned on component state + polling rather than a real backend you designed yourself — schema design, auth you built (not just wired up), automated testing, and CI/CD for a JS/TS full-stack app specifically (as opposed to .NET) are the areas most likely to be thin. This roadmap is weighted accordingly: Stage 1 assumes frontend familiarity and pushes harder on backend/data fundamentals.

Baseline self-check — do this before Stage 1

If you can answer all of these confidently and correctly, skim Stage 1 and move faster. If two or more give you trouble, that's exactly where Stage 1 should slow down for you.

- ? Explain, in order, everything that happens between a user clicking “Log in” on a form and their dashboard rendering — including where the password ever gets checked and where the session/token gets stored.
- ? Write (on paper, no IDE) a SQL query joining three related tables and filtering + sorting the result.
- ? What's the actual difference between `useState`, `useReducer`, and lifting state up — and how would you decide between them for a given piece of state?
- ? What does your React/TypeScript dashboard do if the API call fails mid-poll? If the honest answer is “nothing, it just breaks,” that's a real gap to close in Stage 1.

The 12 Weeks at a Glance

Stage	Focus	Checkpoint
1	Core full-stack fundamentals, properly solidified (frontend, backend API, relational data, real auth)	Spine project v1: full-stack CRUD app with auth, deployed
2	Professional practices — testing, CI/CD, database migrations, PR/review discipline	Spine v2: full test suite + CI pipeline, auto-deploy on merge
3	Systems thinking — caching, realtime, security, performance, containerisation	Spine v3: realtime feature + cache + Dockerised + proper environments
4	Portfolio, communication, AI-assisted development discipline, interview readiness	Spine v4 polished + documented; mock interview; applications go out

Your Spine Project

Rather than four disconnected projects, build one application that grows with you across all four stages. This mirrors how real codebases evolve, and it gives you one strong, deeply-understood centrepiece for your portfolio — you can speak to every architectural decision in it because you made every one of them, at every stage of maturity.

Deliberately pick something outside finance/trading. Your existing portfolio (EquityLens, the algo-trading system) already proves you can do data-heavy, ML-flavoured work. What it doesn't yet prove is that you can own the unglamorous full-stack core — schema design, auth, testing, deployment — that most day-to-day full-stack roles actually run on. This project should fill that specific gap.

Pick one you'll actually stay interested in for 12 weeks. A few that scale well across all four stages:

- A shared kanban/project-tracker (boards, cards, roles, real-time updates when a teammate moves a card)
- A group expense-splitter (shared groups, running balances, settle-up logic — good excuse for real business logic and edge cases)

- A small booking/scheduling system (availability windows, conflicts, notifications)
- A content platform with roles (e.g. a recipe or review site with authors/admins/moderation)

Whichever you choose, resist scope creep. The point isn't the idea — it's demonstrating depth at each stage on the same foundation.

Stage 1 — Weeks 1–3 Core Full-Stack Fundamentals, Solidified

The goal here isn't to learn new things so much as to make sure nothing you're relying on is “magic.” Everything in this stage, you should be able to build without a framework doing it for you first — then use the framework/library with real understanding of what it's saving you from.

Skill targets

- **Frontend:** React fundamentals beyond surface level — component lifecycle with hooks, controlled vs uncontrolled inputs, state colocation (state lives as close as possible to where it's used), custom hooks for shared logic. TypeScript in strict mode — interfaces vs types, generics, discriminated unions for state (e.g. modelling loading/error/success as a type, not three separate booleans).
- **Backend:** Build a REST API from scratch with Node.js (Express or Fastify) — routing, request validation, centralised error-handling middleware, environment-based config (never hard-code secrets or URLs).
- **Data:** Design your own relational schema in Postgres — primary/foreign keys, normalisation up to 3NF, basic indexes on columns you'll actually query by. Use an ORM (Prisma is a good default) once you understand what raw SQL it's generating for you.
- **Auth:** Implement a login flow yourself once — password hashing (bcrypt/argon2), and either sessions or JWTs, deliberately choosing one and being able to justify it — before reaching for an auth-as-a-service library on your next project.

Principles to internalise

- Composition over inheritance — build UI from small, focused components you combine, not deep component hierarchies.
- Unidirectional data flow — state flows down, events flow up; if you find yourself fighting this, that's a sign the state lives in the wrong place.
- REST resources should be nouns, actions should be HTTP verbs — /tasks + POST, not /createTask.
- Statelessness — your API shouldn't need to “remember” anything about a client between requests beyond what's in the token/session.
- Data integrity belongs in the database (constraints, foreign keys), not just in app-level validation — the app can have bugs; the DB is your last line of defence.

Checkpoint — Stage 1

Spine v1: a full-stack CRUD app, your own schema, your own REST API, your own auth flow (not a copy-pasted template), deployed somewhere public (Vercel/Render/Railway are fine for now). No tests yet — that's Stage 2. It should be ugly if it has to be, but it must be real and it must be yours end to end.

Self-check questions

- ? Walk through exactly what your login endpoint does, line by line, including what's stored where and why.
- ? Why did you choose JWT over sessions (or vice versa) for this app — what would make you choose differently next time?
- ? Show me a foreign key constraint in your schema and explain what bug it prevents.
- ? What's the difference between PUT and PATCH, and does your API actually use them correctly?

Stage 2 — Weeks 4–6 Professional Engineering Practices

This is the stage that separates “can build an app” from “can be trusted on a team's codebase.” None of it is glamorous. All of it is what actually gets checked in interviews and probation periods.

Skill targets

- **Testing:** Unit tests (Jest or Vitest) for business logic, integration tests for your API routes (spin up a test DB, hit real endpoints), and at least one end-to-end test (Playwright) covering your core user flow (e.g. sign up → create item → see it persist).
- **CI/CD:** A GitHub Actions pipeline: lint → test → build → deploy on merge to main. You already know this shape from Azure DevOps — the transfer here is mostly syntax, not concept, so lean on that.
- **Collaboration:** Adopt a real branching discipline (feature branches + PRs, even solo), write commit messages that explain why not just what, and practise reviewing your own PRs as if a stranger had to approve them.
- **Data, deeper:** Versioned migrations for every schema change (never edit the DB by hand), and learn to read an EXPLAIN ANALYZE output well enough to spot a missing index. Learn to recognise the N+1 query problem when you see it.
- **API maturity:** Schema validation on every input (zod is a good choice with TypeScript), consistent error response shapes, and basic pagination on any list endpoint.

Principles to internalise

- Test behaviour, not implementation — a good test survives a refactor; a brittle one breaks every time you rename a variable.
- The testing pyramid — lots of fast unit tests, fewer integration tests, a handful of e2e tests. Not the other way round.
- CI should fail fast and fail loud — a red pipeline is doing its job; a green pipeline with no real tests is lying to you.
- Migrations are code — reviewable, versioned, never hand-edited in production.
- Small PRs are a courtesy to your reviewer (and to future-you debugging a regression) — if a PR needs a paragraph to summarise, it probably needs to be two PRs.

Checkpoint — Stage 2

Spine v2: your Stage 1 app now has a real test suite (unit + integration + at least one e2e), a green CI pipeline that runs on every push, and auto-deploys on merge to main. Every schema change from

here on goes through a migration file, not a manual edit.

Self-check questions

- ? What's the N+1 query problem, and can you point to a place in your own code where it could happen?
- ? Walk through your CI pipeline step by step — what does each stage catch, and what would slip through if you removed it?
- ? What would make you request changes on a PR versus just leaving a comment?
- ? Show a migration file and explain what would break if you'd edited the table directly instead.

Stage 3 — Weeks 7–9 Systems Thinking & Production Readiness

This is where you stop thinking “does it work on my machine” and start thinking “what happens under load, under attack, or at 3am when it breaks.” Your trading dashboard's polling loop is actually a good concrete anchor for the realtime section below — you'll be learning why and when you'd swap that pattern out.

Skill targets

- **Realtime & caching:** Add one genuinely realtime feature via WebSockets or Server-Sent Events (compare this deliberately against the setInterval-polling approach you used on the trading dashboard — you'll now understand the trade-off, not just the syntax). Add a caching layer (Redis) for a hot read path, and think through cache invalidation before you add it, not after.
- **Security:** Work through the OWASP Top 10 at a practical level — SQL injection, XSS, CSRF, broken auth — and deliberately try to break your own Stage 1–2 app, then fix what you find. Proper secrets management (environment variables, never committed).
- **Performance:** Code-splitting and lazy loading on the frontend, an actual before/after measurement of a query you optimise with an index, and the discipline of profiling before you touch anything — don't guess at bottlenecks.
- **Infra:** Dockerise the app (one Dockerfile, ideally docker-compose for app + DB together), and do one deploy to a real cloud target with separate dev/staging/prod environment configs — Azure is a reasonable default given your existing familiarity, but any major cloud is fine.

Principles to internalise

- Cache invalidation is the actually-hard part — adding a cache is easy; knowing when to evict it correctly is where bugs live.
- Not everything needs to be realtime — add WebSockets because the feature demands it, not because it's more impressive on a CV.
- Never trust client input — validate and sanitise on the server regardless of what the frontend already checks.
- Defence in depth — no single security measure is your only line of defence.
- Measure before you optimise — a profiler beats intuition every time.

Checkpoint — Stage 3

Spine v3: one realtime feature, one deliberately-added cache with a stated invalidation strategy, a security pass with documented fixes, and the whole thing running in Docker with separate environment configs for dev/staging/prod.

Self-check questions

- ? If a read-heavy endpoint suddenly got 10x traffic, what's your actual first move — and why isn't it just “add a cache” blindly?
- ? Show me one place a malicious user could break your app by sending unexpected input, and how you closed it.
- ? Why WebSockets/SSE for this specific feature and not just faster polling?
- ? What's in your Dockerfile and why does each line need to be there?

Stage 4 — Weeks 10–12 Portfolio, Communication & Job-Readiness

Technically you're close to done by now. This stage is about making that visible and legible to someone who has eight minutes to look at your GitHub before a screening call.

Skill targets

- **AI-assisted development, with judgement:** This is explicitly what employers like the studio in the example JD are screening for — not “can you use Copilot” but “do you know when not to.” Practise reviewing AI-generated code as critically as a colleague's PR, and be ready to explain in an interview where you did and didn't reach for it.
- **Technical communication:** Write a proper README for the spine project — the why, not just the what — and a short case-study write-up of the two or three hardest decisions you made and what you'd do differently now.
- **System design basics:** Be able to whiteboard your own project's architecture from memory, and reason (even roughly) about how you'd scale it if usage grew 100x — you don't need textbook system design depth yet, but you do need to not freeze when asked.
- **Interview mechanics:** A light DSA refresh (arrays, hashmaps, two-pointer patterns — enough for junior-level rounds, not competitive-programming depth), and 3–4 STAR-format stories drawn from real decisions on this project and your existing work.

Principles to internalise

- Judgement over automation — the value you add over an AI tool is knowing when its output is subtly wrong.
- A README explains why a decision was made, not just what the code does — the code already says what it does.
- Two or three deeply-understood projects beat five shallow ones on a CV.
- You should be able to explain your own architecture at two levels: 90 seconds for a non-technical stakeholder, full depth for a senior engineer.

Checkpoint — Stage 4

Spine v4: documented, deployed, with a README and a short case-study write-up. Run one mock technical interview (a friend, a mentor, or a structured self-run mock) against it. Applications go out this week, not “when it feels perfect.”

Self-check questions

- ? Explain your project's architecture in 90 seconds to a non-technical friend, then again in full depth as if to a senior engineer interviewing you.
- ? What would you rebuild differently if you started this project today, and why?
- ? Pick one place you used an AI tool — what would you have done differently if it had given you subtly wrong code, and how would you have caught it?
- ? What's the single weakest part of this project, and what's your plan for it if a live technical review turns onto it?

Appendix A: Reading the Reference Job Description

You chose the general full-stack track rather than the creative-agency track, which is the right call given where you're aiming — but the DixonBaxi JD is still useful as a real, current example of how a studio talks about full-stack seniority. Here's how its specific stack choices map onto the general landscape you're actually training for:

Layer	DixonBaxi JD example	General equivalents you'll meet
Frontend framework	Next.js + React, TypeScript end to end	React (CRA/Vite) or Next.js are the two you'll see most; TypeScript-everywhere is now table stakes, not a bonus
Styling	SCSS Modules	CSS Modules, Tailwind, or styled-components are the common alternatives — pick one and go deep rather than sampling all four
Data / backend	Convex or Supabase (backend-as-a-service)	Traditional Node/Express + Postgres is still the most common baseline in interviews; Supabase/Convex-style BaaS is a productivity layer worth knowing exists once your fundamentals are solid
Realtime	Implicit via Convex/Supabase realtime subscriptions	WebSockets or SSE if you're not using a BaaS with realtime built in
Creative-web (specialised)	React Three Fiber, Three.js, GLSL shaders, Motion	Agency/creative-studio specific — not expected for general full-stack roles; treat as an optional specialisation, not a gap
AI-native	Vercel AI SDK, RAG/agent workflows (bonus)	Increasingly common across the industry, not just creative studios — worth light exposure by Stage 4 regardless of which track you're on

The bigger signal in that JD isn't the specific libraries — it's the shape of seniority they're describing: “know at least one part of this stack deeply enough to have opinions,” judgement about when to use AI tools versus write it yourself, and comfort working directly with designers/stakeholders rather than off a fully-specified ticket. That shape of expectation is now fairly universal across full-stack roles, agency or not — it's the whole reason this roadmap is built around principles and judgement, not just a library checklist.

Appendix B: After Week 12

Being “job-ready” at the end of this plan means: interview-ready for junior/junior-mid full-stack roles, with one deeply-understood project and the vocabulary to discuss trade-offs like an engineer, not a tutorial-follower. It doesn't mean finished learning — nobody is. Two habits worth carrying forward past week 12:

- **Keep one project alive.** A spine project you keep adding small features to for months, at a low simmer, teaches you more than starting a new tutorial every week.
- **Read real code weekly.** Pick one well-regarded open-source repo in your stack and read (not just skim) a PR or module every week — it's the fastest way to absorb conventions you won't get from solo projects.